```
    product = new_product("logo.svg", "image/svg+xml")
    assert_not product.valid?, "image/svg+xml must be invalid"
  end
```

You'll notice that we also added an extra parameter to our `assert` method calls. All of the testing assertions accept an optional trailing parameter containing a string. This will be written along with the error message if the assertion fails and can be useful for diagnosing what went wrong.

Finally, our model contains a validation that checks that all the product titles in the database are unique. To test this one, we need to store product data in the database.

One way to do this would be to have a test create a product, save it, then create another product with the same title and try to save it too. This would clearly work. But a much simpler way is to use Rails fixtures.

Test Fixtures

In the world of testing, a *fixture* is an environment in which you can run a test. If you're testing a circuit board, for example, you might mount it in a test fixture that provides it with the power and inputs needed to drive the function to be tested.

In the world of Rails, a test fixture is a specification of the initial contents of a model (or models) under test. If, for example, we want to ensure that our `products` table starts off with known data at the start of every unit test, we can specify those contents in a fixture, and Rails takes care of the rest.

You specify fixture data in files in the `test/fixtures` directory. These files contain test data in YAML format. Each fixture file contains the data for a single model. The name of the fixture file is significant: the base name of the file must match the name of a database table. Because we need some data for a `Product` model, which is stored in the `products` table, we'll add it to the file called `products.yml`.